

Mining and the Block Header

Rust Miner project · Notes 01 · 6 September 2026

These notes cover what mining actually computes, and the exact structure of the 80 bytes that get hashed. This is the reference for Stage 2 of the roadmap, where the toy string-plus-counter is replaced by a real Bitcoin block header.

1 What mining actually is

Mining is guessing. A miner assembles a block, hashes its 80-byte header, and checks whether the resulting number is below a threshold called the **target**. It almost never is, so the miner changes one field and hashes again. Billions of times per second, by every miner on the network at once.

```
loop {
    header.nonce += 1;
    let h = sha256(sha256(header));
    if h < target {
        broadcast(block);    // you won
        break;
    }
}
```

That is the whole algorithm. Increment, hash, compare. Nothing is being solved and there is no partial progress: a miner that has guessed a trillion times is no closer than one that just started. Every guess is independent.

Why this is worth doing at all

Hashing is hard to do and trivial to check. Producing a hash below the target costs real electricity; verifying someone else's claim costs one hash. That asymmetry is what converts electricity into un-forgable votes, which is what stops an attacker from rewriting history. The rare hash is a receipt for energy burned.

2 The 80-byte header

Six fields, always in this order, always exactly 80 bytes regardless of how many transactions the block contains.

#	Field	Size	What it holds	Miner varies?
1	version	4	Protocol version, plus bits used to signal support for soft forks.	Barely
2	prev_block_hash	32	Hash of the previous block's header. This single field is the entire "chain" in blockchain.	No
3	merkle_root	32	One hash summarising every transaction in the block.	Indirectly
4	timestamp	4	Unix time in seconds, self-reported by the miner.	Yes
5	nBits	4	The target, in a compressed 4-byte encoding.	No
6	nonce	4	A meaningless counter that exists only to be changed.	Yes

$4 + 32 + 32 + 4 + 4 + 4 = 80$ bytes

3 The fields, one at a time

version (4 bytes)

Originally a plain version number. Now most of its bits are used for soft-fork signalling (BIP9): miners flip a bit to say "I support this proposed rule change." When enough blocks in a window signal support, the rule activates. A miner can technically roll a few of these bits as extra search space, but it is noise compared to the nonce.

prev_block_hash (32 bytes)

The double-SHA256 of the previous block's 80-byte header. Completely fixed — it is determined by whichever block is currently at the tip of the chain.

This is the field that makes the chain tamper-evident. Because block N+1 commits to the exact bytes of block N, editing anything in block N changes its hash and leaves block N+1 pointing at something that no longer exists. Note the direction: the **successor** detects the tampering, never the predecessor. A block is protected by what is built on top of it, not by anything beneath it.

merkle_root (32 bytes)

Every transaction in the block is hashed, then hashes are paired and hashed together, repeatedly, until one 32-byte value remains. That is the Merkle root.

It is what keeps the header at a constant 80 bytes no matter whether the block holds one transaction or four thousand. It also means the header commits to the full contents of the block: change any transaction and the root changes, so the header changes, so the block's hash changes and its proof of work is destroyed.

This is the field the miner varies indirectly, and section 4 explains how.

timestamp (4 bytes)

Unix seconds, written by the miner. It is only loosely constrained: it must be greater than the median timestamp of the previous 11 blocks, and no more than 2 hours ahead of network-adjusted time. Blocks can and do appear slightly out of chronological order.

Ordering in Bitcoin comes from the hash links, never from timestamps.

nBits (4 bytes)

The current target, squeezed into 4 bytes. One byte is an exponent, three bytes are a mantissa — essentially scientific notation for a 256-bit number.

It is not an instruction from anyone. Every node independently recomputes what nBits should be every 2016 blocks (roughly two weeks) by measuring how long those blocks actually took, and rejects any block whose nBits disagrees.

nonce (4 bytes)

A counter with no meaning. It affects nothing in the protocol and is ignored by every part of the system except the hash function. It exists purely so that miners have something free to change.

4 bytes gives 2^{32} values, about 4.3 billion — which modern hardware exhausts in well under a second. That is the reason the extranonce exists.

4 The nonce is not enough: extranonce

Once all 4.3 billion nonce values are used up without a win, the miner needs a fresh header. It gets one by changing the **coinbase transaction** — the special transaction the miner writes itself to collect the block reward.

The coinbase carries a free-form data area. Consensus allows its signature script to be **2 to 100 bytes**; BIP34 reserves the first few for the block height, leaving roughly 96 bytes the miner can fill with anything. That spare space is the **extranonce**.

Change one byte of it and the coinbase transaction's hash changes, so the Merkle root changes, so the header is entirely different — and 4.3 billion fresh nonces become available.

Field	Typical size	Chosen by	Purpose
extranonce1	~4 bytes	The pool	Unique per connection, so no two miners in a pool ever grind identical work.
extranonce2	~4–8 bytes	The miner	The miner's own outer counter.

The search is therefore nested:

```
for extranonce in 0.. {           // EXPENSIVE
    rebuild coinbase transaction
    recompute merkle root         //  -> whole new header

    for nonce in 0..2^32 {        // CHEAP
        hash 80 bytes, compare to target
    }
}
```

The asymmetry is the point. Rolling the nonce means rehashing 80 bytes. Rolling the extranonce means rebuilding a transaction and recomputing the entire Merkle tree. So the cheap loop goes inside.

Combined, 2^{32} nonces multiplied by roughly 2^{64} extranonce values gives about 2^{96} distinct headers — vastly more than will ever be needed.

5 Target, difficulty, and “below the target”

A hash is a 256-bit integer. The target is a 256-bit integer. “Below the target” is a plain numeric comparison — nothing clever.

SHA-256 output is spread uniformly across the whole range, so if the target sits at one millionth of the maximum, roughly one in a million guesses lands below it. Lower target, narrower winning band, more work. That is all difficulty is.

Because a small 256-bit number written in fixed-width hex has zeros at the front, this shows up visually as “the hash starts with many zeros.” Same statement, easier to eyeball.

The retarget

Every 2016 blocks, each node measures how long those blocks took and adjusts the target so the next 2016 average 10 minutes each. Faster than expected means hashpower joined, so the target drops. Nobody administers this — every node computes the same number from the same timestamps and independently reaches the same answer.

6 Endianness: the trap in Stage 2

The single most common source of “my hash does not match.” Worth reading twice before writing the serializer.

Field	Stored in the header as	Displayed by explorers as
version, timestamp, nBits, nonce	Little-endian integers	Ordinary decimal or hex numbers
prev_block_hash, merkle_root	Raw 32 bytes, internal order	Byte-reversed from the stored order

So a block hash printed as 0000 . . . a3f1 on a block explorer is stored in the next block's header with its bytes in the opposite order. The leading zeros you see when reading are trailing zeros in memory.

Expect to get this wrong three or four times. That is the stage working as intended — it is what makes the format physical rather than described.

7 Quick reference

Quantity	Value
Header size	80 bytes, always
Hash function	SHA256(SHA256(header)) — applied twice
Nonce space	$2^{32} \approx 4.3$ billion
Extranonce space	$\sim 2^{64}$ in practice; up to ~96 free bytes allowed
Coinbase script limit	2 to 100 bytes (consensus rule)
Retarget interval	2016 blocks, about two weeks
Target block time	10 minutes
Max retarget step	4x up or down, per adjustment
Timestamp rules	> median of previous 11 blocks; < network time + 2 hours

Next: Stage 1 builds the grinding loop against a toy target. Stage 2 replaces the toy input with the real 80 bytes described above.